

mon dataset comprend (nombre_passager, temperature, humidite, mois, jour_semaine, periode_temps(8 periodes dans 24heurs), target c'est nombre_passager entre [0,5] =1 et [6,9]=2, [10,max]=3 j'ai appliqué les quatre algorithmes de classification j'ai reçu les résultats respectivement (regression logistique, naïve bayes, knn, decision tree) --> accuracy (0.7724,0.7525,0.7703,0.7847) precision(0.6850,0.7354,0.7347,0.7039), recall(0.7724,0.7525,0.7703,0.7847), f1-score(0.7248,0.7290,0.7490,0.7421) , expliquer moi en détail les résultats et fournir moi un résumé sur ma distribution

D'accord, analysons en détail vos résultats de classification et votre distribution cible.

1. Analyse de la Distribution Cible

Votre variable cible est une classification du **nombre de passagers** en trois catégories :

- **Classe 1** : [0-5] passagers → **Trafic faible**
- **Classe 2** : [6-9] passagers → **Trafic moyen**
- **Classe 3** : [10-max] passagers → **Trafic élevé**

Interprétation de la distribution (basée sur les résultats) :

Bien que vous n'ayez pas fourni le nombre exact d'instances par classe, les métriques nous donnent des indices forts :

- **La précision est systématiquement inférieure au rappel** pour tous les modèles (sauf Naive Bayes où c'est proche).
- **L'accuracy est relativement élevée** (autour de 77%), ce qui suggère qu'une classe est probablement **majoritaire**.

Hypothèse la plus probable :

La **Classe 1 ([0-5] passagers)** est vraisemblablement la plus représentée dans votre dataset. Cela expliquerait pourquoi :

- Les modèles ont tendance à mieux prédire cette classe

- Le rappel est bon (ils capturent bien la classe majoritaire)
- La précision est plus faible (beaucoup de prédictions pour cette classe, mais incluent des erreurs)

2. Analyse Détaillée des Résultats par Algorithme

A. Decision Tree (Arbre de Décision)

- **Accuracy (0.7847) : MEILLEUR** résultat
- **Rappel (0.7847) : MEILLEUR** → capture bien les instances positives
- **F1-score (0.7421) : 2ème** position

Analyse : L'arbre de décision performe le mieux globalement. Il semble bien capter les règles complexes dans vos données (température, humidité, période, etc.) pour distinguer les niveaux de fréquentation.

B. Logistic Regression (Régression Logistique)

- **Accuracy (0.7724) : 3ème** position
- **Précision (0.6850) : PLUS FAIBLE**
- **F1-score (0.7248) : Plus faible**

Analyse : La régression logistique suppose une relation linéaire entre les variables et la cible. Si vos caractéristiques n'ont pas de relation linéaire claire avec le nombre de passagers, cela explique ses performances moindres.

C. K-Nearest Neighbors (KNN)

- **Accuracy (0.7703) : 4ème** position
- **Précision (0.7347) : 2ème** position
- **F1-score (0.7490) : MEILLEUR**

Analyse : KNN a le **meilleur équilibre** entre précision et rappel (F1-score le plus élevé). Cela signifie qu'il fait le meilleur compromis entre éviter les faux positifs et capturer les vrais positifs.

D. Naive Bayes

- **Accuracy (0.7525) : Plus faible**

- **Précision (0.7354)** : 3ème position
- **F1-score (0.7290)** : 3ème position

Analyse : Naive Bayes suppose l'indépendance conditionnelle des caractéristiques. Dans votre cas, la température, l'humidité, la période etc. sont probablement corrélées, ce qui explique ses performances limitées.

3. Résumé Global et Recommandations

Classement des modèles :

1. **Decision Tree** → Meilleur accuracy et rappel
2. **KNN** → Meilleur équilibre (F1-score)
3. **Logistic Regression** → Performances moyennes
4. **Naive Bayes** → Plus faible performance

Problèmes identifiés :

1. **Déséquilibre de classes** probable
2. **Relations non-linéaires** dans vos données (Logistic Regression performe mal)
3. **Caractéristiques dépendantes** (Naive Bayes performe mal)

Recommandations pour améliorer :

1. Pour gérer le déséquilibre :

- Utiliser l'**oversampling** (SMOTE) ou **undersampling**
- Ajouter des **pénalités de classe** dans les modèles
- Utiliser les métriques **precision-recall curve** plutôt que l'accuracy

2. Pour améliorer les modèles :

- **Random Forest** ou **Gradient Boosting** (gèrent mieux le déséquilibre)
- **Grid Search** pour optimiser les hyperparamètres
- **Feature engineering** (créer de nouvelles variables combinées)

3. Analyse supplémentaire :

- **Matrice de confusion** pour voir quelles classes sont confondues
- **Importance des caractéristiques** pour comprendre les drivers principaux

En résumé : Vous avez des données où la fréquentation faible est probablement majoritaire. L'arbre de décision performe le mieux globalement, mais KNN offre le meilleur équilibre. Le déséquilibre des classes est le principal défi à adresser pour améliorer vos résultats.